INTRODUCTION TO WEB TECHNOLOGIES

CSS Layout & Responsive Design

Study Notes • CAT / Semester Revision

Topics covered

- ✓ Backgrounds and borders
- ✓ Margins, padding and the CSS box model
- ✓ Display properties
- ✓ CSS positioning
- ✓ Flexbox
- ✓ Responsive measurement units
- ✓ Viewport
- ✓ Responsive web design and media queries

Prepared from the IWT notes available on StudyDesk (Unit 4 and Unit 5, with related viewport material from Unit 2). Explanations and examples are organized for quick learning and exam revision.

StudyDesk: aditya-verma-06.github.io/studydesk/

1. CSS Backgrounds

The CSS **background** properties control the background effects of an HTML element. Backgrounds may use a solid colour or an image.

Important background properties

Property	Purpose / example
background-color	Sets the background colour. Example: background-color: lightblue;
background-image	Uses an image as the background. Example: background-image: url('paper.gif');
background-repeat	Controls image repetition. Common values: repeat, repeat-x, repeat-y, no-repeat.
background-position	Sets the initial position of the image: center, top, bottom, left, right.
background-attachment	Controls whether the background image scrolls or remains fixed.

Example

```
body {
  background-color: lightblue;
  background-image: url("paper.gif");
  background-repeat: no-repeat;
  background-position: right top;
  background-attachment: fixed;
}
```

Opacity: The `opacity` property controls transparency. Values range from 0.0 (fully transparent) to 1.0 (fully opaque). A value of 0.5 means 50% opacity.

```
img { opacity: 0.5; }
img:hover { opacity: 1.0; }
```

Remember: By default, a background image repeats to cover the element. Use `background-repeat: no-repeat;` when repetition is not wanted.

2. CSS Borders

A **border** is drawn around an HTML element. It can be customized using border style, colour, and width.

Property	What it controls	Example
border-style	Appearance of the border	solid, dotted, dashed, double, none
border-color	Border colour	red, #174e43, rgb(0,0,0)
border-width	Thickness	1px, 2px, 5px
border-radius	Rounds the corners	border-radius: 10px;
border	Shorthand for border width, style and colour	border: 2px solid green;

Example

```
.card {
  border: 2px solid green;
  border-radius: 10px;
}
```

Common border styles include **solid** (continuous line), **dashed** (short strokes), **dotted** (dots), and **double** (two lines).

3. Margin and Padding

Padding

Padding is the space **between the content and the border**. It is inside the border. Padding is transparent unless a background is applied.

Margin

Margin is the space **outside the border**, separating an element from neighbouring elements. Margin is transparent.

Feature	Padding	Margin
Location	Inside the border	Outside the border
Purpose	Space around content	Space between elements
Background	Element background extends through padding	Transparent area outside border
Can use auto?	Not for centering in the usual way	margin: auto can horizontally centre a fixed/limited-width block

Shorthand values

```
/* Same value on all four sides */
margin: 10px;
padding: 10px;

/* Vertical | horizontal */
margin: 10px 20px;

/* Top | right | bottom | left (clockwise) */
margin: 10px 20px 30px 40px;

/* Individual sides */
padding-top: 12px;
margin-bottom: 20px;
```

Margin auto: A common way to centre a block horizontally is to give it a width and set its left and right margins to auto.

```
.box {
  width: 50%;
  margin: auto;
}
```

Margin collapse: In some vertical layouts, adjacent top and bottom margins combine into one margin rather than being added. For example, 20px and 30px may collapse to 30px (the larger value).

4. CSS Box Model

The CSS box model describes the rectangular box that wraps around every HTML element. It consists of four parts, from inside to outside:

Part	Meaning
Content	The actual text, image, or other content.
Padding	Space around the content, inside the border.
Border	Line surrounding the padding and content.
Margin	Transparent space outside the border.

Order to memorize: Content → Padding → Border → Margin.

Width calculation (default content-box)

If width is 300px, padding is 50px on both left and right, and border is 15px on both sides:

```
Total width = content width + left/right padding
             + left/right border
            = 300 + 50 + 50 + 15 + 15
            = 430px

Margin is outside this calculation.
```

box-sizing

The `box-sizing` property controls how the declared width and height are calculated.

Value	Meaning
content-box (default)	Declared width/height apply to content; padding and border are added outside.
border-box	Declared width/height include content, padding and border.

```
* {
  box-sizing: border-box;
}
```

This global rule makes sizing easier when building responsive layouts.

5. CSS Display Properties

The `display` property controls how an element participates in layout. HTML elements have default display behaviour, commonly block or inline.

Value	Behaviour
block	Starts on a new line and takes the available width. Examples: div, p, headings.
inline	Does not start a new line; takes only the width needed. Examples: span, a.
inline-block	Flows inline, but width, height, padding and margin can be applied like a block.
none	Removes the element from the document layout; it takes no space.
flex	Makes the element a flex container for arranging its direct children.
grid	Makes the element a grid container for row-and-column layout.

Important difference: `display: none` removes the element and its layout space. `visibility: hidden` hides it but keeps its space.

```
span { display: inline; }
.card { display: block; }
.container { display: flex; }
.grid { display: grid; }
.hidden { display: none; }
```

6. CSS Positioning

The `position` property specifies how an element is positioned. The top, right, bottom and left properties can then adjust its location.

Value	Meaning / behaviour
static	Default. Element follows normal document flow; offsets do not position it.
relative	Remains in normal flow but is shifted relative to its original position.
absolute	Positioned relative to its nearest positioned ancestor; removed from normal flow.

Value	Meaning / behaviour
fixed	Positioned relative to the viewport; stays in the same place while the page scrolls.
sticky	Acts like relative until a scroll threshold is reached, then sticks like fixed within its containing area.

Examples

```
.relative {
  position: relative;
  left: 30px;
}

.parent { position: relative; }
.child {
  position: absolute;
  top: 10px;
  right: 0;
}

.fixed-button {
  position: fixed;
  bottom: 10px;
  right: 10px;
}

header {
  position: sticky;
  top: 0;
}
```

Sticky vs fixed: Sticky initially occupies its place in the document and sticks after reaching a threshold. Fixed is attached to the viewport from the start and does not reserve its original layout space.

7. Flexbox

Flexbox (Flexible Box Layout) is a CSS layout module used to arrange, align and distribute space among items in a container. It is especially useful for one-dimensional layouts (a row or a column) and smaller components.

Flex container and flex items

• Flex container: the parent element with `display: flex`.

• Flex items: the direct children of the flex container.

• Main axis: the direction in which items are arranged.

• Cross axis: the axis perpendicular to the main axis.

```
.container {
  display: flex;
}
<div class="container">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

By default, flex items are arranged in a row.

Flex container properties

Property	Values / purpose
flex-direction	row (default), row-reverse, column, column-reverse. Sets the main direction.
flex-wrap	nowrap (default), wrap, wrap-reverse. Controls whether items move onto additional lines.

Property	Values / purpose
flex-flow	Shorthand combining flex-direction and flex-wrap; e.g. row wrap.
justify-content	Aligns/distributes items along the main axis: flex-start, flex-end, center, space-between, space-around, space-evenly.
align-items	Aligns items along the cross axis: stretch, center, flex-start, flex-end, baseline.
align-content	Controls distribution of multiple flex lines along the cross axis, especially when wrapping.

Flex item properties

Property	Purpose
order	Changes the visual order of an item. Items are sorted by ascending order value.
flex-grow	Controls how much an item grows to use available space. Default: 0.
flex-shrink	Controls how much an item shrinks when space is insufficient. Default: 1.
flex	Shorthand for flex-grow, flex-shrink and flex-basis.

Responsive Flexbox example

```
.flex-container {
  display: flex;
  flex-direction: row;
}

/* On smaller screens, stack the items */
@media (max-width: 800px) {
  .flex-container {
    flex-direction: column;
  }
}
```

Flexbox vs Grid: Flexbox is generally suited to one-dimensional layouts (one row or one column). Grid is designed for two-dimensional layouts (rows and columns).

8. Responsive Measurement Units

The StudyDesk Unit 5 notes list %, em, rem, px, vh and vw as common CSS measurement units.

Unit	Meaning	Example
%	Relative to another value, often the containing element's size.	width: 80%;
em	Relative to the current element's font size (as described in the course notes).	font-size: 2em;
rem	Relative to the root element's font size, usually html.	font-size: 1.5rem;
px	CSS pixel; a predictable CSS length unit.	font-size: 16px;
vh	1vh equals 1% of viewport height.	height: 50vh;
vw	1vw equals 1% of viewport width.	width: 50vw;

Quick memory: vw = viewport width; vh = viewport height; rem = root font size; % = relative size.

9. Viewport

The **viewport** is the visible area of a web page in the browser. It varies by device and is usually smaller on a phone than on a desktop.

Use the viewport meta tag so the page uses the device's screen width and starts at the intended zoom level:

```
<meta name="viewport"
      content="width=device-width, initial-scale=1.0">
```

• `width=device-width` makes the page width follow the device's screen width.

• `initial-scale=1.0` sets the initial zoom level.

• Avoid large fixed-width elements that cause horizontal scrolling.

• Use relative widths and media queries so content fits different screens.

10. Responsive Web Design (RWD)

Responsive Web Design is an approach using HTML and CSS to make a webpage look good on different devices. Content can resize, shrink, enlarge, move or adapt for desktops, tablets and phones. It is a design approach, not a programming language.

Key practices

• Design content to fit the viewport and avoid unnecessary horizontal scrolling.

• Avoid relying on one fixed screen width.

• Use flexible or relative dimensions such as percentages.

• Use responsive images, for example `max-width: 100%; height: auto;`.

• Use CSS media queries to apply different styles at different screen sizes.

• A responsive grid may use columns whose widths shrink and expand with the browser.

Responsive images

```
img {
  max-width: 100%;
  height: auto;
}
```

The HTML `<picture>` element can provide different image sources for different browser widths using media conditions on `<source>` elements.

11. CSS Media Queries

A media query is a CSS technique that applies a block of styles only when a specified condition is true. It uses the `@media` rule.

Basic syntax

```
@media only screen and (max-width: 600px) {
  body {
    background-color: lightblue;
  }
}
```

The example applies the style when the screen width is 600px or less.

Common media-query conditions

Condition	Use
max-width	Apply styles at or below a specified width.
min-width	Apply styles at or above a specified width.
orientation: landscape	Apply styles when the viewport is wider than it is tall.

Condition	Use
orientation: portrait	Apply styles when the viewport is taller than it is wide.

Examples

```
/* Hide an element on small screens */
@media (max-width: 600px) {
  .example { display: none; }
}

/* Different text size at different widths */
@media (min-width: 601px) {
  .example { font-size: 40px; }
}
@media (max-width: 600px) {
  .example { font-size: 20px; }
}
```

Breakpoints and mobile-first design

A breakpoint is a screen width at which the layout changes to suit the available space. The course notes demonstrate 800px and 600px examples. Mobile-first design means designing for smaller screens first, then adding or changing styles for larger screens.

```
/* Mobile-first: base styles suit small screens */
.container { display: flex; flex-direction: column; }

/* Add a row layout on wider screens */
@media (min-width: 768px) {
  .container { flex-direction: row; }
}
```

12. Quick Revision: Differences to Remember

Compare	Remember
Margin vs padding	Margin is outside the border; padding is between content and border.
display:none vs visibility:hidden	none removes layout space; hidden keeps the space.
relative vs absolute	relative shifts from its normal position; absolute uses the nearest positioned ancestor.
fixed vs sticky	fixed stays at viewport position; sticky sticks after a scroll threshold.
Flexbox vs Grid	Flexbox is mainly one-dimensional; Grid is two-dimensional.
vh vs vw	vh is based on viewport height; vw is based on viewport width.
min-width vs max-width	min-width sets a lower limit; max-width sets an upper limit.

13. Small Practice Snippets

A. Centre a box and add spacing

```
.box {
  width: 60%;
  margin: auto;
  padding: 20px;
  border: 2px solid black;
}
```

B. Make three items align in the centre

```
.container {
  display: flex;
```

```
  justify-content: center;
  align-items: center;
  gap: 12px;
}
```

C. Change to a column on a phone

```
.container {
  display: flex;
  flex-direction: row;
}
@media (max-width: 600px) {
  .container { flex-direction: column; }
}
```

D. Responsive image

```
img {
  max-width: 100%;
  height: auto;
}
```

14. Exam-Focused Checkpoints

• Box model order: content, padding, border, margin.

• Padding is inside the border; margin is outside.

• Block starts on a new line; inline does not.

• display:none removes an element from layout; visibility:hidden preserves its space.

• position: static is the default.

• position: fixed is relative to the viewport.

• position: absolute is relative to the nearest positioned ancestor.

• Flexbox parent uses display:flex; direct children become flex items.

• justify-content works along the main axis; align-items works along the cross axis.

• flex-direction values: row, row-reverse, column, column-reverse.

• Viewport meta tag: width=device-width, initial-scale=1.0.

• Media queries use @media and conditions such as max-width and min-width.

• 1vh = 1% viewport height; 1vw = 1% viewport width.

Source coverage note: Backgrounds, borders, box model, margin/padding, display and positioning are based mainly on IWT Unit 4. Flexbox, viewport, responsive units, responsive design and media queries are based on IWT Unit 5. The notes condense the source material and add short explanatory examples for revision.

Study material accessed from the user's StudyDesk website: Unit 4_PPT IWT and 5.Unit 5. Dr. Vikas Lamba (Flex, RWD, Animation), with viewport terminology cross-checked against Unit 2.